

Case Studies III — The Finale

The last six systems — geospatial, sync, concurrency, money — and a capstone that ties it all together.

-
- 65 Design Ride-Sharing (Uber)** — matching riders and drivers in real time
 - 66 Design File Storage & Sync (Google Drive)** — chunking, dedup, and sync
 - 67 Design Ticket Booking (Ticketmaster)** — concurrency without double-booking
 - 68 Design a Payment System** — correctness, idempotency, and reconciliation
 - 69 Design a Nearby-Places Service** — geospatial indexing
 - 70 Capstone: Putting It All Together** — the method, end to end
-

The final case studies apply every pattern from Parts I-IX; Chapter 70 closes the book by assembling the whole method.

CHAPTER 65

Design Ride-Sharing (Uber)

Part X · Case Studies — matching riders and drivers in real time.

WHAT THIS CASE STUDY SHOWS

- Tracking millions of moving locations in real time.
- Geospatial indexing to find nearby drivers fast.
- Combining real-time streams, matching, and state machines.

THE CORE CHALLENGE

Ride-sharing must answer “which drivers are near this rider *right now*?” over millions of constantly-moving vehicles — and do it in milliseconds. That is a **real-time geospatial** problem, not a normal database query.

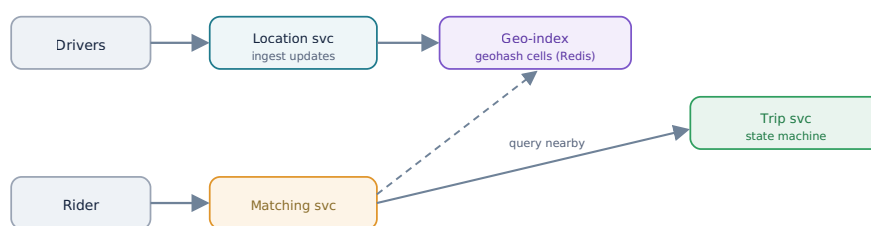
65.1 Requirements

Functional: drivers stream their location; a rider requests a ride; the system finds and matches a nearby driver; both track the trip live. **Non-functional:** low-latency matching, very high write rate (location updates), high availability, and correctness for the trip lifecycle.

65.2 Estimation

Millions of active drivers each sending a location update every few seconds → **millions of writes per second** — a firehose. Matching queries must scan only nearby drivers, not all of them, so the location index is the crux (Chapters 53, 69).

65.3 High-level design



location updates feed a geo-index; matching queries nearby cells; the trip runs as a state machine

65.4 Deep dive: geospatial indexing

KEY INSIGHT — INDEX LOCATIONS INTO GRID CELLS

You can’t scan every driver, so map each location to a **grid cell** using a scheme like **geohash** (or an S2/quadtrees). A geohash turns latitude/longitude into a short string where nearby places share a prefix, so “drivers near me” becomes “drivers in my cell and its neighbors” — a fast lookup instead of a full scan. Keep this index in **memory** (Redis) because it’s updated millions of times a second; the constant location firehose is absorbed

by a streaming ingestion layer. The trip itself is a **state machine** (requested → matched → en route → complete) needing strong consistency, while location data is fine slightly stale.

65.5 Trade-offs

Grid cell size trades precision against query cost. Location freshness vs write load is tunable (update every few seconds, not continuously). Matching also weighs ETA, surge pricing, and fairness. The trip/payment path needs strong consistency; the location layer favors availability and speed (Chapter 20).

Key takeaways

- **Geospatial indexing** (geohash/quadtrees) turns “nearby” into a fast cell lookup.
- Absorb the **location firehose** with an in-memory index and streaming ingestion.
- Run the **trip as a state machine** (strong consistency); keep location data cheap and slightly stale.

CHAPTER 66

Design File Storage & Sync (Google Drive)

Part X · Case Studies — chunking, de-duplication, and multi-device sync.

WHAT THIS CASE STUDY SHOWS

- Splitting files into chunks for efficient upload and storage.
- De-duplication and delta sync to save bandwidth.
- Keeping many devices consistent and resolving conflicts.

THE CORE CHALLENGE

File sync must upload and store huge files efficiently, avoid re-transferring unchanged data, and keep **multiple devices** in sync — even after offline edits that conflict. Chunking and metadata are the keys.

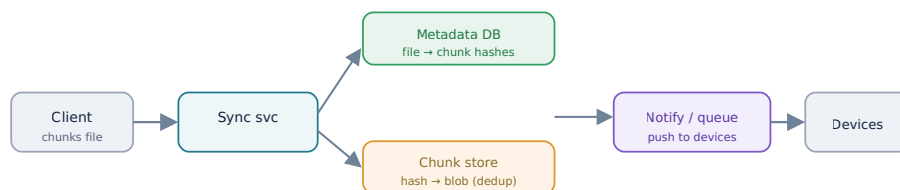
66.1 Requirements

Functional: upload/download files; sync across devices; share; version history. **Non-functional:** durable storage, efficient transfer (don't re-upload unchanged data), reasonable sync latency, and conflict handling.

66.2 Estimation

Billions of files, exabytes of storage. Uploads/downloads dominate bandwidth; many files are **duplicates** (the same attachment stored by thousands of users), so de-duplication yields massive savings (Chapters 49, 53).

66.3 High-level design



files split into content-hashed chunks; metadata maps files→chunks; changes pushed to other devices

66.4 Deep dive: chunking, dedup, and delta sync

KEY INSIGHT — STORE CONTENT-ADDRESSED CHUNKS ONCE

Split each file into **chunks** and identify every chunk by a **hash of its content**. Store the chunk once, keyed by that hash; if two files (or two users) share a chunk, it's stored a single time — **de-duplication** for free. A file becomes just an ordered list of chunk hashes in the

metadata DB. When a file changes, only the **changed chunks** upload (**delta sync**), saving huge bandwidth. Other devices are told which chunks changed and fetch just those. This content-addressed design gives dedup, delta sync, and easy versioning at once.

66.5 Trade-offs and conflicts

Offline edits on two devices cause **conflicts**; resolve with versioning (keep both as “conflicted copies”) or last-writer-wins with history, since true merge is often impossible. Sync is eventually consistent (Chapter 21). Chunk size trades dedup granularity against metadata overhead. Encryption at rest protects stored chunks (Chapter 43).

Key takeaways

- Split files into **content-hashed chunks**; store each chunk once → automatic **de-duplication**.
- Files are **metadata** (ordered chunk hashes); changed files upload only **changed chunks** (delta sync).
- Handle multi-device **conflicts** via versioning; sync is eventually consistent.

CHAPTER 67

Design Ticket Booking (Ticketmaster)

Part X · Case Studies — concurrency without double-booking.

WHAT THIS CASE STUDY SHOWS

- Preventing two people from buying the same seat.
- Handling extreme, bursty contention on popular events.
- Choosing strong consistency where money and inventory demand it.

THE CORE CHALLENGE

When tickets go on sale, thousands of people fight for the **same seats** in the same instant. The system must guarantee a seat is sold **exactly once** — a hard concurrency and consistency problem, the opposite of the eventually-consistent feeds we've seen.

67.1 Requirements

Functional: browse events and seats; hold a seat during checkout; confirm purchase; release the hold if payment fails or times out. **Non-functional:** **no double-booking** (strong consistency), fairness, and surviving massive traffic spikes on hot events.

67.2 Estimation

Browsing is read-heavy (cacheable), but a popular on-sale creates a **thundering herd** on a tiny set of seats — extreme write contention concentrated on a few rows. That contention, not overall volume, is the challenge (Chapters 5, 53).

67.3 Deep dive: locking a seat safely



only one request wins the lock on a seat — the other is refused, so no double-booking

KEY INSIGHT — MAKE “CLAIM A SEAT” AN ATOMIC, EXCLUSIVE OPERATION

The seat must be claimed **atomically** so exactly one buyer wins. Options: a **database transaction** with row locking (Chapter 14) — conditionally flip the seat from **available** to **held** only if still available; or a **distributed lock** (Chapter 24) for the seat. Use a **temporary hold** with a short expiry (a reservation) so the seat is locked during checkout and auto-released if payment doesn't complete — preventing seats from being frozen forever. This is firmly a **CP** choice (Chapter 20): correctness over availability, because selling a seat twice is unacceptable.

| 67.4 Handling the spike

Absorb the on-sale rush with a **virtual waiting room / queue** that admits users at a controlled rate (rate limiting, Chapter 35), protecting the booking core from the thundering herd. Browsing/seat maps are cached and can be slightly stale; the **hold and purchase** path is strongly consistent. Optimistic concurrency (compare-and-set) works well when most attempts don't actually collide.

| Key takeaways

- Claim a seat with an **atomic, exclusive** operation (row lock or distributed lock) — exactly one winner.
- Use **time-limited holds** that auto-release so seats aren't frozen by abandoned checkouts.
- This is a **CP** system — correctness over availability; shield the core with a **waiting room**.

CHAPTER 68

Design a Payment System

Part X · Case Studies — correctness, idempotency, and reconciliation.

WHAT THIS CASE STUDY SHOWS

- Guaranteeing money is never lost or double-charged.
- Idempotency and the saga pattern across external providers.
- Auditability and reconciliation as first-class requirements.

THE CORE CHALLENGE

With money, **correctness is everything**: never double-charge, never lose a payment, and always be able to prove what happened. Because payments cross unreliable networks and external providers, **idempotency** and careful state handling are non-negotiable.

68.1 Requirements

Functional: charge a payment; handle success/failure; issue refunds; record every transaction. **Non-functional**: strong consistency and correctness, a complete **audit trail**, resilience to provider/network failures, and security (Chapters 43–44).

68.2 Estimation

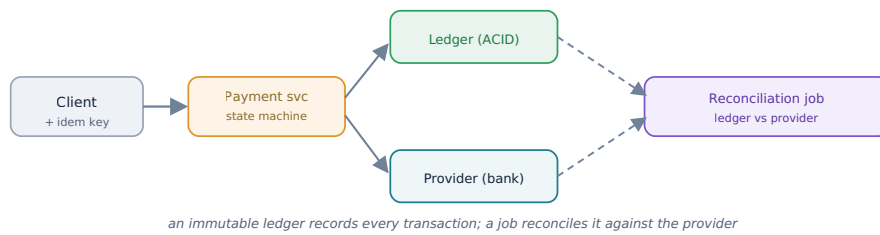
Volume matters less than **correctness** here — even modest QPS demands zero tolerance for lost or duplicated charges. Every transaction must be durably recorded and reconcilable against the external payment provider.

68.3 Deep dive: idempotency and reliable state

KEY INSIGHT — IDEMPOTENCY KEYS MAKE RETRIES SAFE

Networks fail mid-payment (Chapter 19), so clients and services **retry** — and a naive retry could charge twice. The fix: every payment carries a client-generated **idempotency key** (Chapter 25). The payment service records that key with the result; a retry with the same key returns the *original* outcome instead of charging again. Model each payment as a **state machine** (initiated → pending → succeeded/failed) persisted at every step, so a crash can resume safely. Because a payment spans your service, a ledger, and an external provider, coordinate it as a **saga** with compensations (Chapter 38) — e.g. a refund compensates a charge — rather than a distributed transaction.

68.4 Ledger, reconciliation, and consistency



Keep an **immutable, append-only ledger** (double-entry) as the source of truth — a natural fit for event sourcing (Chapter 33) and a complete audit trail. A periodic **reconciliation** job compares your ledger against the provider's records to catch any discrepancy. This is a **strongly consistent** (CP) system throughout — availability is sacrificed before correctness.

Key takeaways

- **Idempotency keys** make payment retries safe — the same key never charges twice.
- Model payments as a persisted **state machine**; coordinate across providers with a **saga**, not 2PC.
- Keep an **immutable ledger** and **reconcile** against the provider; choose **strong consistency** always.

CHAPTER 69

Design a Nearby-Places Service

Part X · Case Studies — geospatial indexing for “what’s around me?”

WHAT THIS CASE STUDY SHOWS

- Answering proximity queries efficiently over millions of places.
- Geohash and quadtree indexing in depth.
- A read-heavy design that’s the counterpart to Uber’s moving points.

THE CORE CHALLENGE

“Find restaurants within 2 km” can’t scan every place and compute distances — that’s far too slow at scale. You need a **spatial index** that narrows the search to a small nearby region first. Unlike Uber (Chapter 65), the points are mostly **static**, so the index is read-optimized.

69.1 Requirements

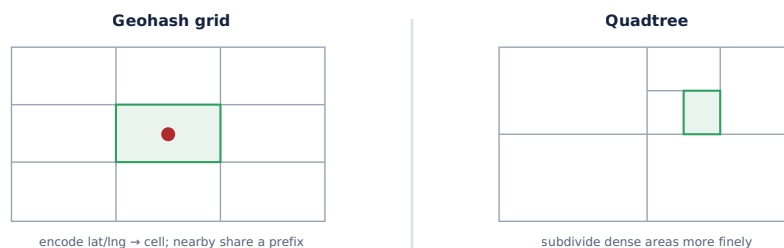
Functional: given a location and radius, return nearby places, optionally filtered/ranked.

Non-functional: low-latency reads at high volume, mostly static data (places change rarely), high availability.

69.2 Estimation

Hundreds of millions of places, but each query touches only a small area. Read QPS is high; writes (new/updated places) are rare, so the index can be heavily precomputed and cached (Chapter 53).

69.3 Deep dive: geohash vs quadtree



KEY INSIGHT — INDEX SPACE INTO CELLS YOU CAN LOOK UP

Two classic approaches. **Geohash** encodes latitude/longitude into a short string so that nearby locations share a **common prefix**; a proximity query becomes “fetch places whose geohash matches my cell and its 8 neighbors,” then filter by exact distance — a fast prefix lookup instead of a global scan. A **quadtree** recursively subdivides space, splitting dense areas (a city) into finer cells and leaving sparse areas coarse, which balances how many places sit in each cell. Either way you turn an expensive distance computation over everything into a cheap lookup over a tiny candidate set, then refine.

69.4 Trade-offs

Cell size trades result completeness against query cost (too coarse returns too many candidates; too fine needs many neighbor cells). Boundary cases (a place just across a cell edge) require checking neighbors. Because places are mostly static, the index is precomputed and **cached** aggressively and replicated for read scale — the mirror image of Uber’s write-heavy moving index (Chapter 65).

Key takeaways

- Use a **spatial index** (geohash or quadtree) to convert “nearby” into a cheap cell lookup, then refine by exact distance.
- **Geohash** = prefix-shared grid cells; **quadtree** = adaptive subdivision for uneven density.
- Static data → **precompute, cache, and replicate** for read-heavy scale.

CHAPTER 70

Capstone: Putting It All Together

Part X · Case Studies — the whole method, end to end.

WHAT THIS CLOSING CHAPTER DOES

- Assembles the entire book into one coherent method.
- Walks the social-platform capstone through the full framework.
- Leaves you with the mindset of a system designer.

WHERE WE STARTED, AND WHERE WE ARE NOW

We began with a single CPU and a single request. Seventy chapters later, you can reason about globally-distributed systems serving billions of users. The journey wasn't about memorizing architectures — it was about building the **judgment** to assemble the right pieces for a given problem, and to explain why.

70.1 The method, in one view

Every design in this book followed the same path (Chapter 52): **clarify requirements**, **estimate scale** (Chapter 53), sketch an **API** and a **high-level design**, **deep-dive** the hard part, address the **non-functional** concerns, and reason through the **trade-offs** (Chapter 54). The case studies proved the payoff: nearly every “new” system was a fresh **combination** of patterns you already knew.

70.2 The recurring toolkit

Need	Reach for
Scale reads	Caching (Ch8), CDN (Ch8), read replicas (Ch16), read models (Ch33)
Scale writes / data	Sharding (Ch17), consistent hashing (Ch18), queues (Ch28)
Decouple & absorb spikes	Message queues (Ch28), event-driven (Ch32)
Cross-service consistency	Sagas (Ch38), not distributed transactions (Ch24)
Survive failure	Redundancy (Ch34), retries/backoff (Ch36), circuit breakers (Ch37), degradation (Ch40)
“Nearby” / real-time	Geospatial index (Ch65/69), WebSockets (Ch31), fan-out (Ch57)
Save memory / space	Bloom filters & sketches (Ch39), content-addressed chunks (Ch66)

70.3 The capstone, assembled

YOUR SOCIAL PLATFORM — THE WHOLE PICTURE

Across the book you built one system end to end: requirements and estimates (Chs 1, 5, 53); a tiered, stateless app layer behind load balancers (Chs 6, 7, 10) with an API gateway (Ch9); a polyglot data layer — relational core, cache, graph for the follow network, wide-column for the activity stream (Chs 11–18) — replicated and sharded; a feed built with fan-out (Ch57) over queues (Ch28); real-time notifications and messaging (Chs 31, 59); microservices connected by gRPC and a service mesh (Chs 26, 30) with sagas for cross-service operations (Ch38); reliability from redundancy, rate limiting, circuit breakers, and graceful degradation (Chs 34–40); security throughout (Chs 41–44); deployed on containers with CI/CD, multi-region, and full observability (Chs 45–50). Every choice traced back to a requirement.

70.4 The thesis, one last time

KEY INSIGHT — DESIGN IS REASONED TRADE-OFFS UNDER CONSTRAINTS

There is no perfect architecture — only the one best suited to *your* requirements, scale, and constraints. Mastery is knowing the patterns, judging which fit, and **explaining why**, clearly and honestly (Ch54). Every chapter here was, at heart, a trade-off: consistency vs availability, latency vs durability, simplicity vs scalability, cost vs reliability. Internalize the trade-offs and you can design systems you’ve never seen before.

70.5 Where to go next

Keep building — nothing teaches like operating a real system under load. Read engineering blogs and post-mortems (real failures are the best teachers). Revisit the case studies and design variations. And practice **explaining** your designs aloud, because a design only exists once you can communicate it. You now have the foundations of a staff engineer; the rest is reps.

Final takeaways

- One repeatable **method** handles any design problem: requirements → scale → design → deep dive → trade-offs.
- Most systems are **combinations** of a modest set of patterns — composition beats novelty.
- System design is the discipline of **well-reasoned trade-offs under constraints**, communicated clearly.
- You began at a single request; you can now reason about systems for a billion. Keep building.

THE END — AND THE BEGINNING

This completes **System Design — From First Principles to Staff Engineer, Volume I**. Seventy chapters, ten parts, one method. Thank you for going the whole distance — now go design something.